# Lab 6 - Network Simulator 3 Introduction

# Motivation: (Network Analysis Techniques)

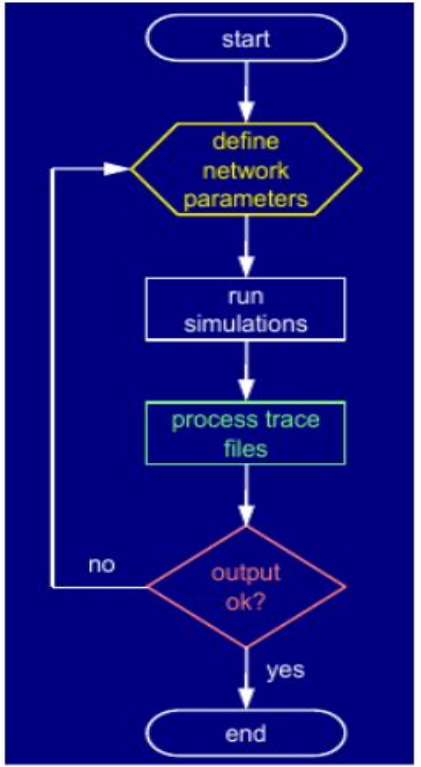
- **Analytical Approaches**
  - Mathematical analysis / modeling of systems
- **Simulations**
  - Model the system at abstract level via software
  - Various network simulators exist (eg. NS3, OpenNet, QualNet)
- **Emulations (eg. Run Windows on MAC)**
  - HW component that behave like real system (eg. Mininet)
- **Measurements**
  - Active (eg. ping, traceroute) or passive (eg. wireshark)
- **Experimentations**
  - Experiment on a testbed for realism (eg. GENI, PlanetLab, OrbitLab, NAMS Lab)

# Motivation for Network Simulations

- **Goals**
  - Build software simulation model of networking systems
  - To analyze/study/improve/develop network archs & protocols
- **Reasons**
  - Real systems are expensive, complex, unavailable
- **Advantages**
  - Relatively easy to set up, deploy and instrument
  - Reproducibility, scalability
- **Disadvantages**
  - Simplified view of complex interactions
  - Could be immensely misleading

# Network Simulation Flowchart

- Define topology, protocols/algos, (channel/traffic) models, flows
- Specify initial values, start/end times
- Run simulations for many seeds
- Process output /trace files
- Measure avg. network performance: throughput, goodput, delay, jitter, loss, fairness, etc
- If not happy, repeat above steps again!



# Scripting and Gnuplot

## ❑ Special Variables:

```
echo "First argument: $0"
echo "Second argument: $1"
echo "Third argument: $2"
echo "Fourth argument: $3"
echo "Fifth argument: $4"
echo "All arguments: $@"
echo "Number of arguments given to the executable: $#"
echo "Exit status of the last command: $?"
echo "Process number of the current shell: $$"
echo "All the arguments: $*"
~
```

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ vim script6.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script6.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script6.sh 1 2 3 4 5
First argument: ./script6.sh
Second argument: 1
Third argument: 2
Fourth argument: 3
Fifth argument: 4
All arguments: 1 2 3 4 5
Number of arguments given to the executable: 5
Exit status of the last command: 0
Process number of the current shell: 3364651
All the arguments: 1 2 3 4 5
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

## ❑ Different methods of evaluating and expression:

```
read n
let x=$n+1
echo "$x"
~
```

```
read n
x=$((n+1))
echo "$x"
~
```

```
read n m
x=`expr '(' $n + 1 ')' / $m`
echo "$x"
~
```

```
read n
x=`expr $n + 10`
echo "$x"
~
```

```
read n m
x=`expr \( $n + 1 \) / $m`
echo "$x"
```

- **Conditional Statements:**

```
If [ expression ]
then
  Statement 1
elif [ expression ]
then
  Statement 2
else
  Statement 3
fi
```

- **Test Expression Evaluation:**
  - ✔ n1 -eq n2 : tests if n1 == n2
  - ✔ n1 -ne n2 : tests if n1 != n2
  - ✔ n1 -lt n2 : tests if n1 < n2
  - ✔ n1 -le n2 : tests if n1 <= n2
  - ✔ n1 -gt n2 : test if n1 > n2
  - ✔ n1 -ge n2 : test if n1 >= n2
  - ✔ if either n1 or n2 is not a number then test fails.

arindam@NAMS-LAB-NTN-401-System-1: ~/Desktop/Arindam Thesis/Simulation S

```
echo -n "Enter your age: "
read n
if [ $n -le 10 ]
then
        echo "This child is not allowed to drive."
elif [ $n -le 18 ]
then
        echo "This child is allowd to use bicycle."
else
        echo "He is allowed to drive car."
fi
~
```

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ vim script5.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script5.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script5.sh
Enter your age: 17
This child is allowd to use bicycle.
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

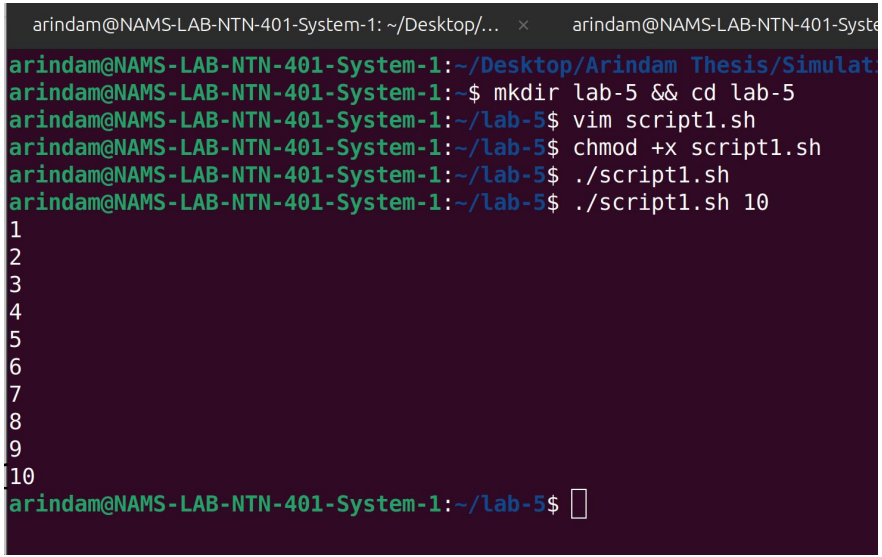
- **For Loop syntax:**

```
for (( initialization; termination condition; update ))
do
  Statement
done
```

Output:
1
2
3
.
.
.
N

- **Example of for loop:**

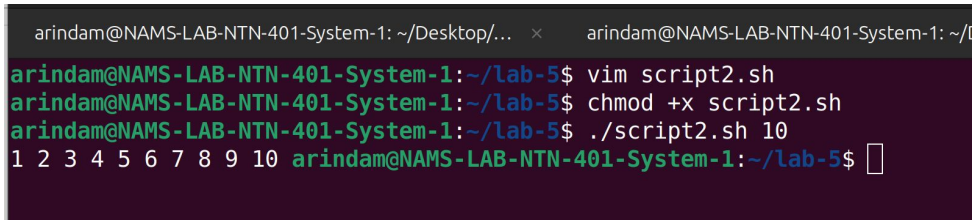
```
N=$1 #read N can also be performed
for (( i = 1; i <= N; i++ ))
do
  echo “$i”
done
```



**Output:**
**1 2 3 … N**

- **Example of for loop:**

```
N=$1 # read N can also be performed
for (( i = 1; i <= N; i++ ))
do
  echo –n “$i ”
done
```



- **Example of for loop (Alternative Approach):**

```
N=$1 # read N can also be performed
for i in $(seq 1 $N)
do
  echo –n “$i ”
done
```

**Output:**
**1 2 3 … N**

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ vim script3.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script3.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script3.sh 10
12345678910arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

- **Printing list of given arguments:**

```
for i in $1 $2 $3 $4 $5
do
  echo –n “$i ”
done
```

**Output:**
**Argument1 argument2 … arguement3**

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ vim script4.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script4.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script4.sh 5 4 10 20
541020arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

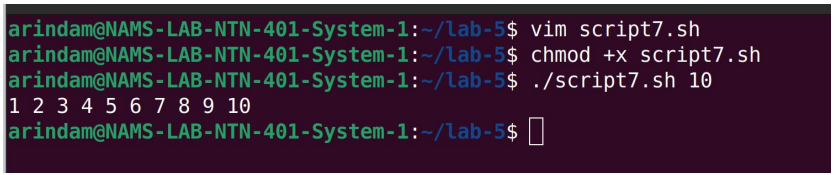
- **While loop syntax:**

```
while [ condition ]
do
  Statement
done
```

- **Example of while loop:**

```
N=$1 # read N can also be performed
i=1
while [ $i -le $N ]
do
  echo -n “$i ”
  ((i++))
done
echo
```

**Output:**
**1 2 3 … N**



- **Example of while loop:**

```
N=$1 #read N can also be performed
i=1
while true
do
  echo -n "$i ”
  ((i++))
  if [ "$i" -gt "$N" ]
  then
        break
  fi
done
echo
```

**Output:**
**1 2 3 … N**

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script8.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script8.sh 8
12345678
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

- **Switch case:**

```
echo -n "Enter the choice(1-4): “
read n
case $n in
  1)
        echo "Statement 1“
        ;;
  2)
        echo "Statement 2“
        ;;
  3)
        echo "Statement 3“
        ;;
  4)
        echo "Statement 4“
        ;;
  *)
        echo "Default Statement“
        ;;
esac
```

**Input: 2**
**Output: Statement2**

**Input: 8**
**Output: Default Statement**

- **Switch case:**

```
echo -n "Enter the choice(1-4): “
read n
case $n in
  1)
        echo "Statement 1“
        ;;
  2)
        echo "Statement 2“
        ;&
  3)
        echo "Statement 3“
        ;;
  4)
        echo "Statement 4“
        ;;
  *)
        echo "Default Statement“
        ;;
esac
```

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ chmod +x script9.sh
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script9.sh
Enter the number: 2
Statement 2
Statement 3
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script9.sh
Enter the number: 1
Statement 1
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script9.sh
Enter the number: 10
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$
```

**Input: 2**
**Output: Statement2**
**Statement3**

- **Switch case:**

```
echo -n "Enter the choice(1-4): “
read n
case $n in
  1)
        echo "Statement 1“
        ;;
  2)
        echo "Statement 2“
        ;&
  3)
        echo "Statement 3“
        ;;
  “NAMS”)
        echo "Statement 4“
        ;;
  *)
        echo "Default Statement“
        ;;
esac
```

```
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script9.sh
Enter the number: NAMS
Statement 4
arindam@NAMS-LAB-NTN-401-System-1:~/lab-5$ ./script9.sh
Enter the number: nams
Default Statement
```

**Input: 2**
**Output: Statement2**
**Statement3**

- **Automated compilation and execution of C program file over different inputs:**
  - Example:

```
arindam@ARINDAM:~$ cat hello.c
#include<stdio.h>
#include<stdlib.h>

void main(int argc, char* argv[]){
        if(argc<2)
        {
                printf("Input not provided!!!");
                exit(1);
        }
        int a = atoi(argv[1]);
        printf("Your entered number is: %d\n",a);
}
arindam@ARINDAM:~$ ./demo.sh
Executing for loop 0:
Your entered number is: 12
Executing for loop 1:
Your entered number is: 13
Executing for loop 2:
Your entered number is: 41
Executing for loop 3:
Your entered number is: 20
Executing for loop 4:
Your entered number is: 55
arindam@ARINDAM:~$
```

```
#! /bin/bash

array=(12 13 41 20 55)
gcc hello.c -o hello
for i in ${!array[@]}
do
        echo "Executing for loop $i:"
        ./hello ${array[$i]}
done
```

## Set terminal

- The terminal defines the output format or device for plots.

Examples:

- **Interactive window** (default: qt or wxt)
- **Image files** (png, jpeg)
- **Document formats** (pdfcairo, eps)

- By default is qt (qt cross-platform interactive terminal)
  - set terminal qt
  - set terminal qt 1 // Opens a separate interactive window with ID = 1.

- *Change terminal*

```
set terminal png          # change terminal
set output "plot.png"     # plot filename
plot sin(x)                  # plot function
unset output                 # file close
```

- **Saving figure in GNU-plot:**
  - To save a figure in gnu-plot, you need to specify the terminal type (Output format) before your plot.
  - General Procedure:
    - set terminal pngcairo size 800,600 enhanced font 'Arial,10’

      set output 'output.png’

      plot sin(x)

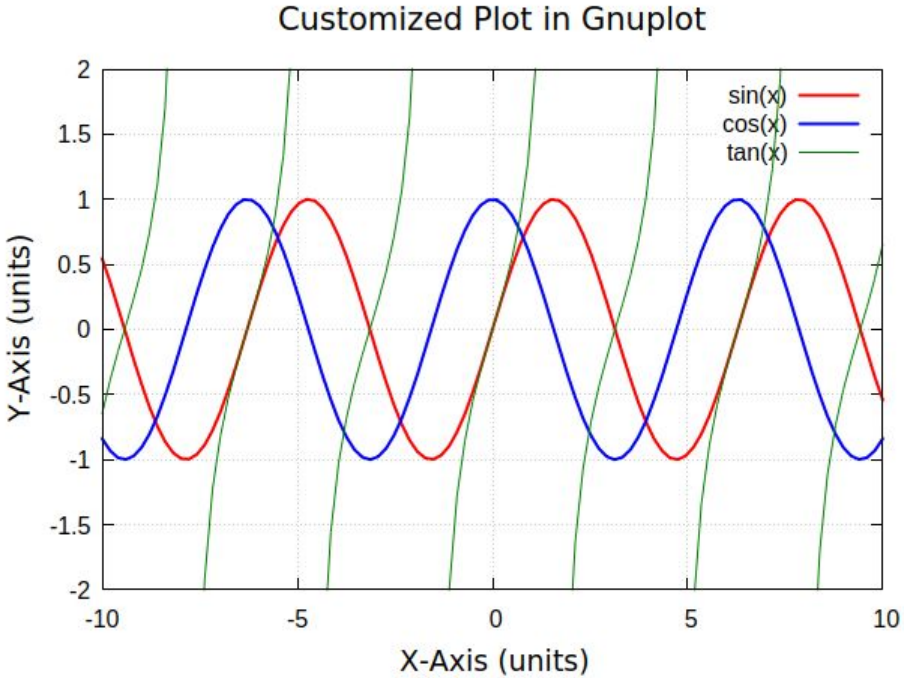
      unset output
  - Things to remember:
    - Always use output command before the plot command.
    - Always unset output after plotting to flush the file and avoid corruption.

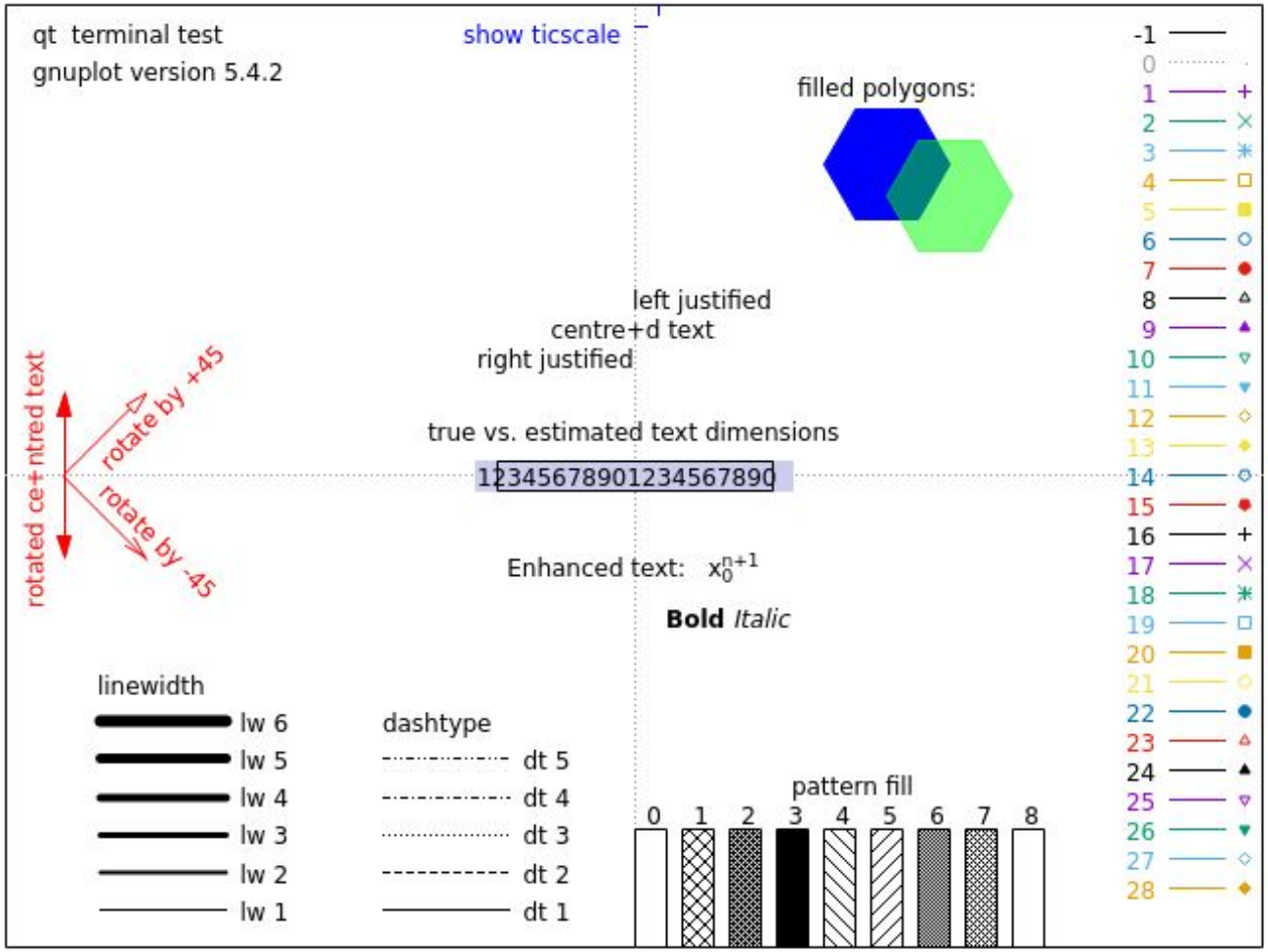
```
# Demo Gnuplot Script: Customizing Plot Appearance
set terminal qt font "Arial,12"          size 800,600                  # Set output terminal
# Axis ranges
set xrange [-10:10]       # Fix x range
set yrange [-2:2]         # Fix y range

# Margins
set lmargin 10
set rmargin 5
set tmargin 3
set bmargin 4

# Labels and Title
set xlabel "X-Axis (units)" font "Times-Bold,14"
set ylabel "Y-Axis (units)" font "Times-Italic,14"
set title  "Customized Plot in Gnuplot" font "Helvetica-BoldItalic,16"

# Grid lines for clarity
set grid
set key top right    //legend
# Plot with customized styles
plot sin(x) lt 1 lc rgb "red"   lw 2 title "sin(x)", \
     cos(x) lt 2 lc rgb "blue"  lw 2 title "cos(x)", \
     tan(x) lt 3 lc rgb "#008000" lw 1 title "tan(x)"
pause -1 "Press Enter to close"
```




qt terminal test
gnuplot version 5.4.2
show ticscale
filled polygons:
left justified
centre+d text
right justified
rotated ce+ntred text
rotate by +45
rotate by -45
true vs. estimated text dimensions
12345678901234567890
Enhanced text: x0n+1
Bold Italic
linewidth
lw 6
lw 5
lw 4
lw 3
lw 2
lw 1
dashtype
dt 5
dt 4
dt 3
dt 2
dt 1
pattern fill
0 1 2 3 4 5 6 7 8
-1
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28

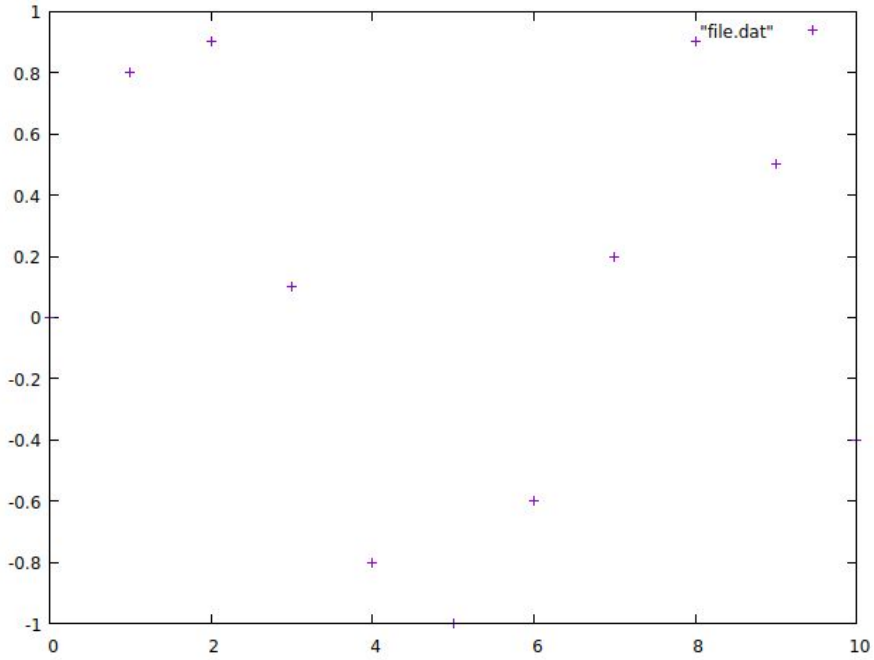
- **ASCII-Datafiles in multi-column format:**

file.dat

```
# Comments can be placed here
x0  y0  z0   ...
x1  y1  z1   ...
.    .   .   ...
xN  yN  zN   …
```

- To plot a multi-column datafile using the 1st column for the abscissa and the 2nd column as the ordinate, use **:** plot “file.dat” using 1:2
- Add a second plot using 1st (=x) and 3rd (=z) columns: replot “file.dat” using 1:3
- If the “using” keyword is not specified, 1st and 2nd columns are assumed: plot “file.dat”

# plot “file.dat” using 1:2




| # X | Y1 | Y2 |
|---|---|---|
| 0 | 0.0 | 1.0 |
| 1 | 0.8 | 0.6 |
| 2 | 0.9 | 0.1 |
| 3 | 0.1 | -0.5 |
| 4 | -0.8 | -0.9 |
| 5 | -1.0 | -0.2 |
| 6 | -0.6 | 0.7 |
| 7 | 0.2 | 0.9 |
| 8 | 0.9 | 0.3 |
| 9 | 0.5 | -0.7 |
| 10 | -0.4 | -1.0 |

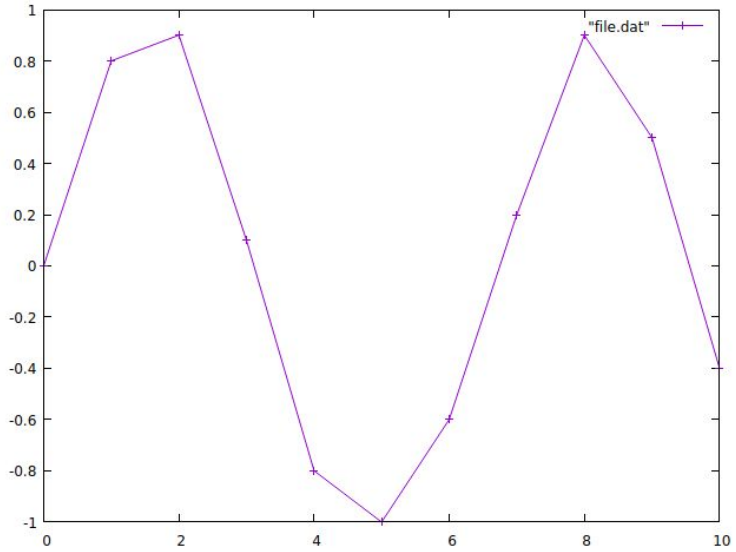
file.dat

- Data file separator: By default, gnuplot treats tabs and white spaces as field value separator.
- If we want to specify the field value separator manually, we have to use this command: **set datafile separator “<separator_name>"**

- **When plotting datafiles, gnuplot uses symbols and join symbol with lines:**

  plot “file.dat” using 1:2 with lines

- plot “file.dat” using 1:2 with linespoints




```
# X   Y1    Y2
0     0.0   1.0
1     0.8   0.6
2     0.9   0.1
3     0.1  -0.5
4    -0.8  -0.9
5    -1.0  -0.2
6    -0.6   0.7
7     0.2   0.9
8     0.9   0.3
9     0.5  -0.7
10   -0.4  -1.0
```

file.dat

**Plotting a barplot:**

- set style data histograms : Instructs GNUplot to treat the data as histogram, rather than a line or scatter plot.
- set style fill pattern 4 border -1 : Pattern fills the bar with predefined pattern, border -1 disables the black border for each bar.
- set boxwidth 0.6 : Sets the relative width of the bars.
- plot 'item1.dat' using 2:xtic(1) title "Item 1 Stock prices" lc rgb "red"

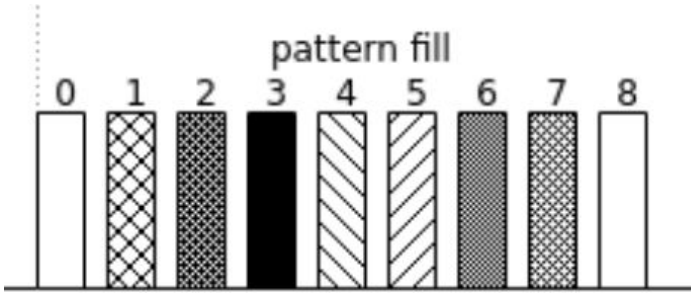
  [ 2:xtic(1), Column 2 is used for the height of the bars (the expenses). Column 1 is used for the X-axis labels (the month names)].

**Different border setting:**

| Command | Meaning |
| --- | --- |
| border -1 | No border at all (completely removes the outline) |
| border 0 | Draws only the **outline (border)** of the bars, but no top/bottom lines |
| border 1 *(default)* | Draws a full border: all 4 sides of the bar |

## ❑ Style for Bar:

```
set style fill empty                          # hollow boxes
set style fill solid 1.0 border               # solid fill, with border (default color)
set style fill solid 0.6 noborder          # semi-dense fill, no border
set style fill transparent solid 0.5     # 50% transparent fill (if terminal supports it)
set style fill pattern 2 border 1 lc rgb "black"  # pattern fill with black border
```



**Example Code:**

```
set terminal pdfcairo size 20cm,14cm font "Times New Roman,12"
set output "monthly_expenses.pdf"
set title "Monthly Expenses"

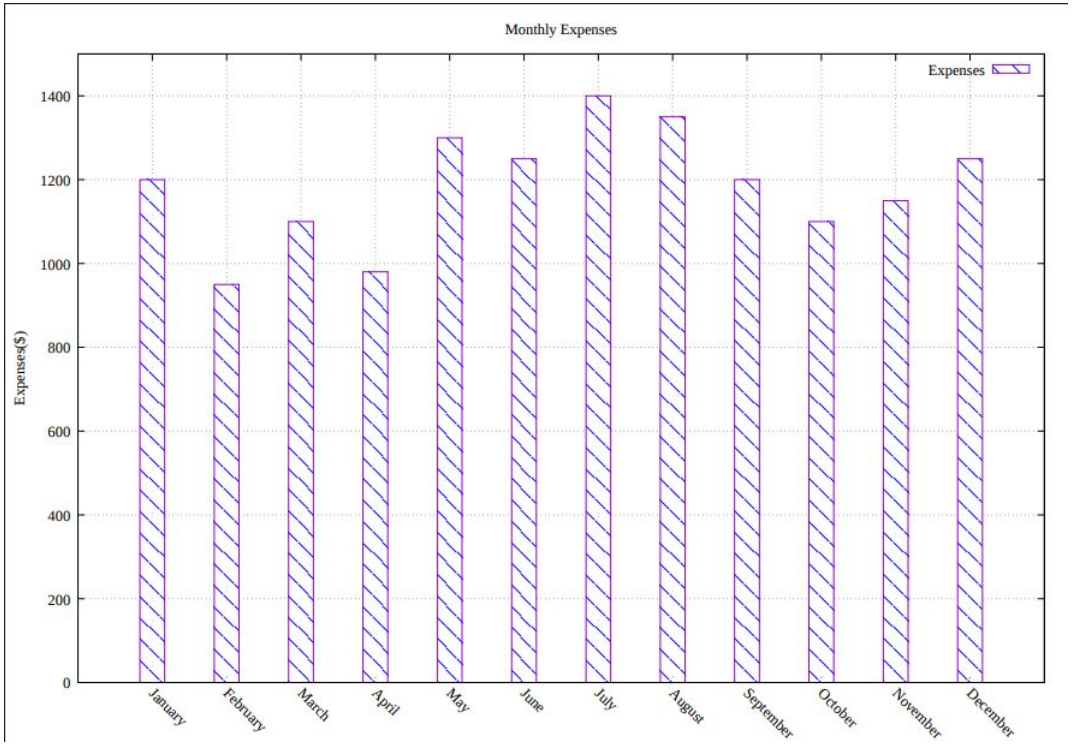
set style data histogram
set style fill pattern 4 border 1
set boxwidth 0.8

set xtics rotate by -45
set xlabel "Month"
set ylabel "Expenses ($)"
set yrange [100:170]

set grid
plot "expenses.dat" using 2:xtic(1) title "Expenses" lc rgb "blue"

unset output
unset terminal
```

# Output:

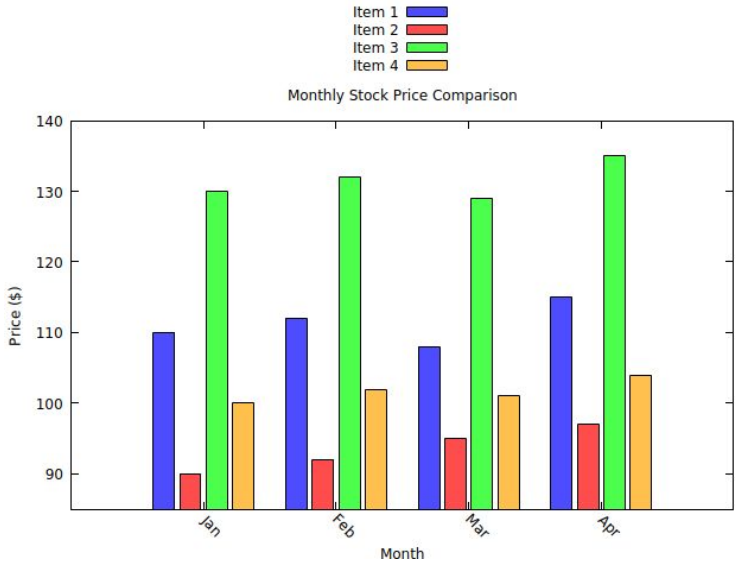

Monthly Expenses
Expenses
Expenses($)
0
200
400
600
800
1000
1200
1400
January
February
March
April
May
June
July
August
September
October
November
December

# ❑ Multibar

```
set style data histograms
set style histogram cluster gap 1
set style fill solid 0.7 border -1
set boxwidth 0.9 relative

set title "Monthly Stock Price Comparison"
set xlabel "Month"
set ylabel "Price ($)"
set key outside top center
set xtics rotate by -45

plot "stock.dat" using 2:xtic(1) title "Item 1" lc rgb "blue", \
    "" using 3 title "Item 2" lc rgb "red", \
    "" using 4 title "Item 3" lc rgb "green", \
    "" using 5 title "Item 4" lc rgb "orange"
```




| Month | Item1 | Item2 | Item3 | Item4 |
|---|---|---|---|---|
| Jan | 110 | 90 | 130 | 100 |
| Feb | 112 | 92 | 132 | 102 |
| Mar | 108 | 95 | 129 | 101 |
| Apr | 115 | 97 | 135 | 104 |

# NS-3

# What is NS (or NS-2/3)?

- NS is a discrete-event network simulator for internet systems
  - Protocol design, multiple levels of abstraction
  - Written in C++, with bindings available for python
- NS has a companion network animator called nam
  - Hence, has been called the nsnam project (www.nsnam.org)

# NS-3: Introduction

1. NS-3 is a free, open source packet-level discrete event network simulator available under GNU GPLv2 license for research, development, and use.
2. Technical Goals
    a. Build and maintain a simulation core aligned with the needs of modern networking research
    b. Help to improve the technical rigor of network simulation practice
3. NS-3 Timeline
    a. NS-1 1990s
    b. NS-2 1996
    c. NS-3 Core development 2006-08 (Tom Henderson and Mathieu Lacage)
    d. NS3.1 June 2008
    e. Now Generally new release every Quarterly (fix bugs and new features, etc..)
    f. NS3.42 May 29, 2024 (Recent Stable Release) You can install developer version to get access to latest modules like 5G module, etc...

# Building NS-3

As of the most recent ns-3 release (ns-3.42), the following tools are needed to get started with ns-3:

| Prerequisite | Package/version |
|---|---|
| C++ compiler | clang++ or g++ (g++ version 9 or greater) |
| Python | python3 version >=3.8 |
| CMake | cmake version >=3.13 |
| Build system | make, ninja, xcodebuild (XCode) |
| Git | any recent version (to access ns-3 from GitLab.com) |
| tar | any recent version |
| bunzip2 | any recent version |

# Building NS-3

Lots of ways to get ns-3 (https://www.nsnam.org/docs/tutorial/html/getting-started.html) but follow the steps below to pull the latest version **closer**, because it does get **older** :)

- wget https://www.nsnam.org/release/ns-allinone-3.44.tar.bz2
- tar xjf ns-allinone-3.44.tar.bz2
- cd ns-allinone-3.44
- ./build.py --enable-examples --enable-tests
- What are you waiting for…? (Yup, it's that easy :))
- Copy sample-simulator.cc from src/core/examples and place it in scratch to execute it because whatever happens in ns-3, happens inside scratch (GOLDEN RULE)

# Building NS-3 (2nd Method)

Lots of ways to get ns-3 (https://www.nsnam.org/docs/tutorial/html/getting-started.html) but follow the steps below to pull the latest version **closer**, because it does get **older** :)

- wget https://www.nsnam.org/release/ns-allinone-3.44.tar.bz2
- tar xjf ns-allinone-3.44.tar.bz2
- cd ns-allinone-3.44/ns-3.44
- ./ns3 --build-profile=debug --enable-tests --enable-examples **or** ./ns3 --build-profile=optimized --enable-tests --enable-examples
- ./ns3 build

# Building NetAnim

- cd netanim-3.109
- cmake -B build
- cd build
- make

Opening NetAnim:

- cd ns-allinone-3.44/netanim-3.109/build/bin
- ./netanim

# Building NS-3

**If you've got disk space and a stable internet connection, ease off the entire process by running the following commands, it might alleviate some of the post-installation error troubleshooting:**

- apt install -y g++ cmake ninja-build ccache libgsl-dev libgtk-3-dev libboost-dev wget git python3 python3-pip
- apt install -y automake bc bison flex gawk libc6 libc6-dbg libdb-dev libssl-dev libpcap-dev vim rsync gdb
- apt install -y mercurial indent libsysfs-dev
- pip3 install requests distro

# Hello World NS-3 Program

This is basic C++, except:

1) We are using methods/classes defined in 'ns3' namespace

2) The object 'cmd' is an instance of the CommandLine C++ class.

CommandLine exists in C++ namespace 'ns3'.

CommandLine objects process command-line arguments.

```
#include <iostream>
#include "ns3/command-line.h"

using namespace ns3;

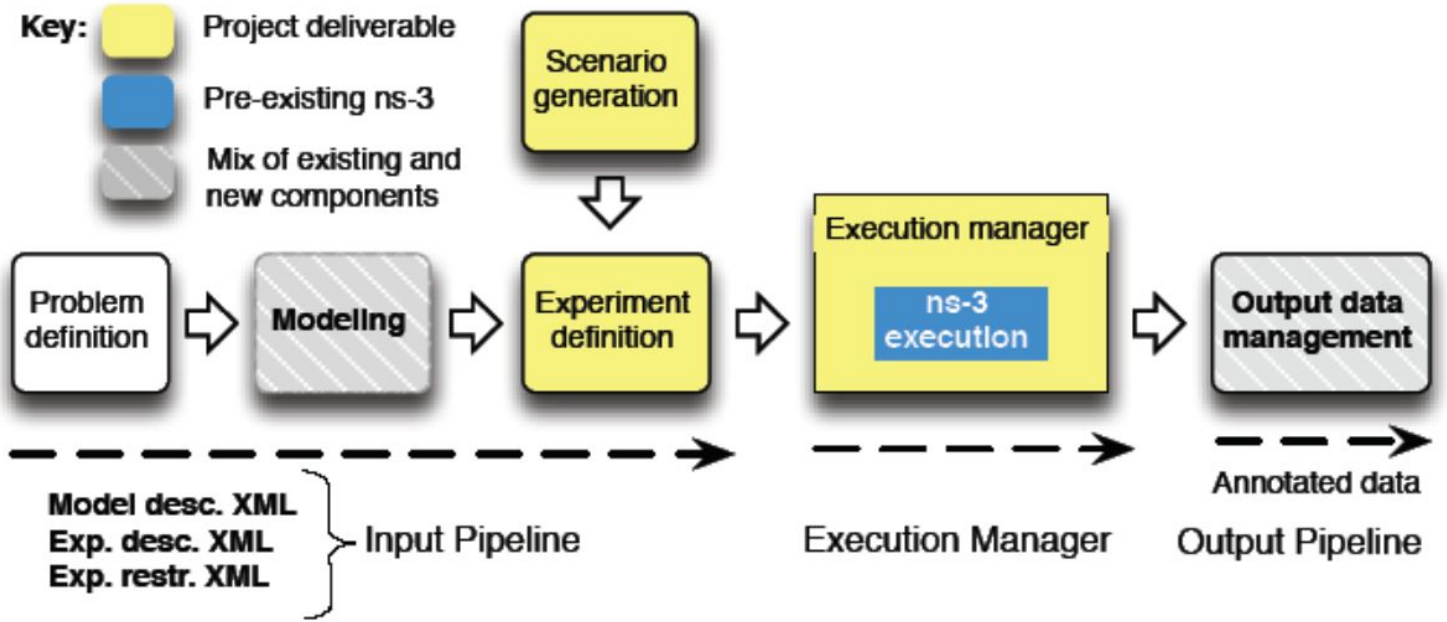
int main (int argc, char *argv[])
{
  std::string language = "English";
  std::string phrase;

  CommandLine cmd;
  cmd.AddValue ("language", "Specify language", language);
  cmd.Parse (argc, argv);

  if (language == "English")
    {
      phrase = "Hello world";
    }
  else if (language == "Italian")
    {
      phrase = "Ciao mundo";
    }
  else
    {
      phrase = "That language is not spoken here";
    }

  std::cout << phrase << std::endl;
}
```

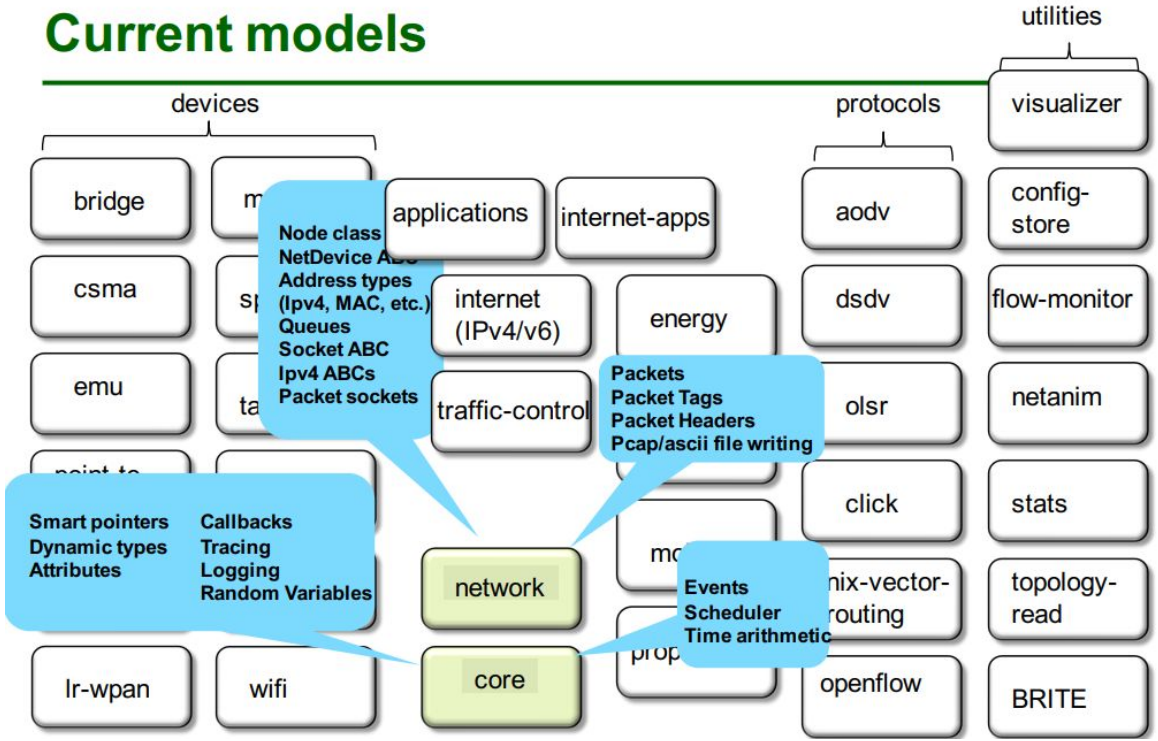
# Framework for NS-3



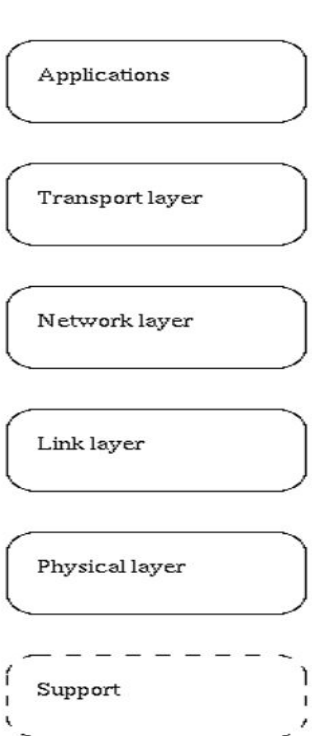
# NS-3 Modules

- A set of related classes, examples, and tests, that can be combined together into an ns-3 module so that they can be used with existing ns-3 modules and by other researchers
- All modules are under src directory
- Wired Network Modules
  - Point-to-Point, csma, Network, Internet, etc
- Wireless Network Modules
  - wifi, wimax, lte, olsr, aodv, dsdv, dsr, mesh, sixlowpan, spectrum, propogation, mobility, etc
- Generic modules
  - Core, energy, flow-monitor, stats, etc
- src/internet/model by far remains the most important directory

# NS-3 Modules

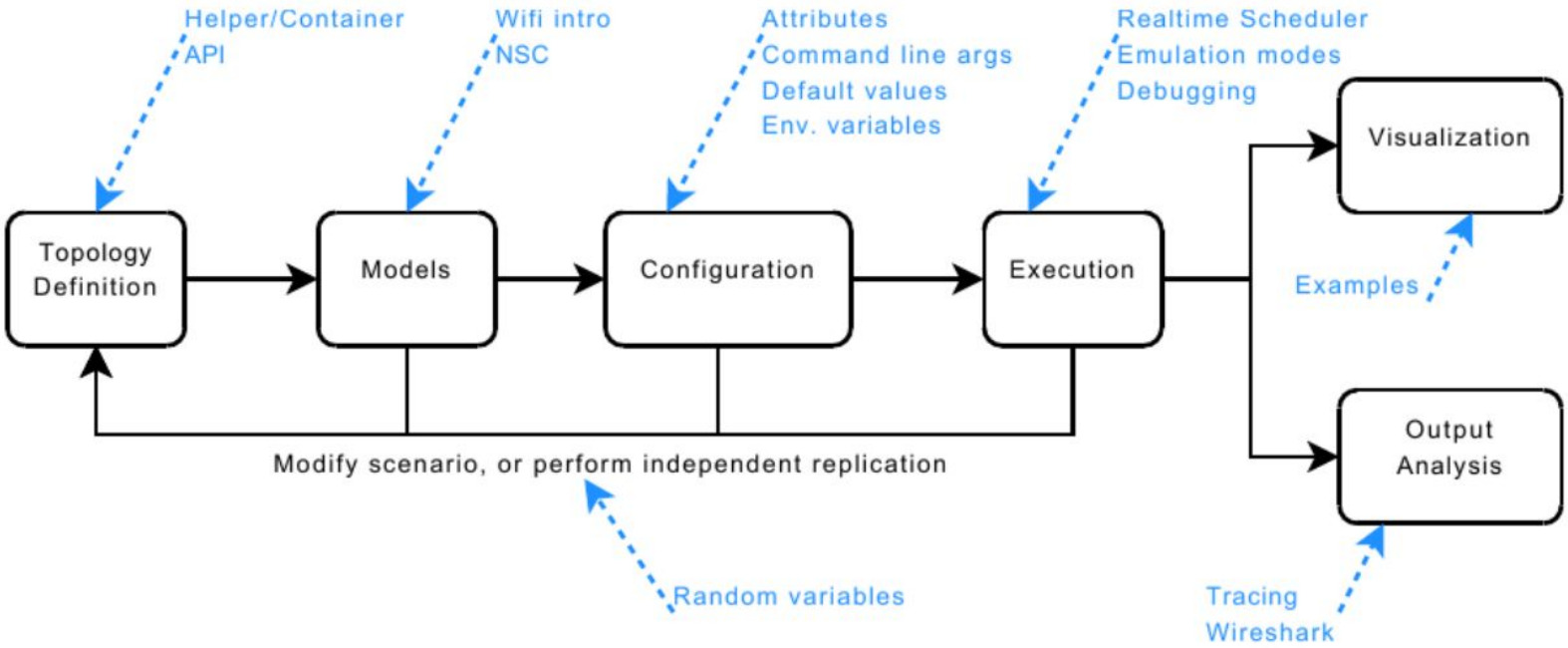

Current models
utilities
devices
protocols
visualizer
bridge
csma
emu
lr-wpan
wifi
applications
internet-apps
internet (IPv4/v6)
energy
traffic-control
network
core
aodv
dsdv
olsr
click
openflow
config-store
flow-monitor
netanim
stats
topology-read
BRITE
Node class
NetDevice
Address types
(Ipv4, MAC, etc.)
Queues
Socket ABC
Ipv4 ABCs
Packet sockets
Packets
Packet Tags
Packet Headers
Pcap/ascii file writing
Smart pointers
Dynamic types
Attributes
Callbacks
Tracing
Logging
Random Variables
Events
Scheduler
Time arithmetic

# NS-3 Models



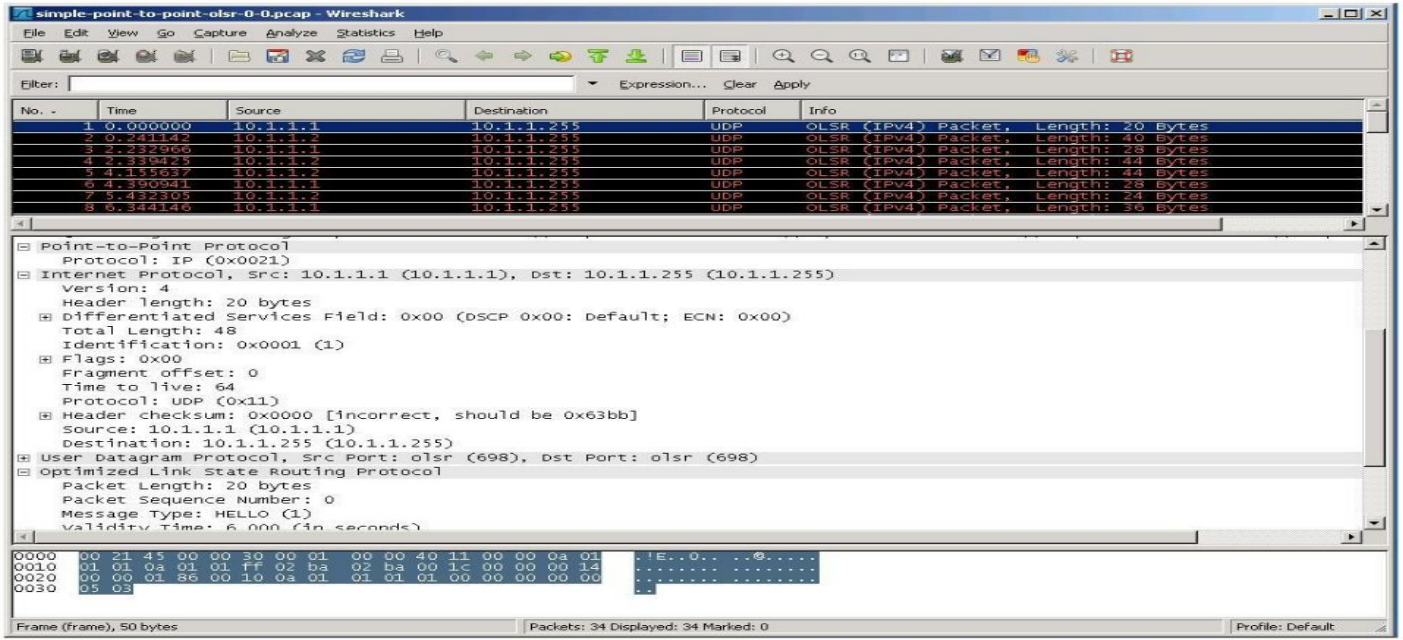

| | Existing core ns-2 capability | Existing ns-3 |
|---|---|---|
| Applications | ping, vat, telnet, FTP, multicast FTP, HTTP, probabilistic and trace-driven traffic generators, webcache | OnOffApplication, asynchronous sockets API, packet sockets |
| Transport layer | TCP (many variants), UDP, SCTP, XCP, TFRC, RAP, RTP<br>Multicast: PGM, SRM, RLM, PLM | UDP, TCP |
| Network layer | Unicast: IP, MobileIP, generic dist. vector and link state, IPinIP, source routing, Nixvector<br>Multicast: SRM, generic centralized<br>MANET: AODV, DSR, DSDV, TORA, IMEP | Unicast: IPv4, global static routing<br>Multicast: static routing<br>MANET: OLSR |
| Link layer | ARP, HDLC, GAF, MPLS, LDP, Diffserv<br>Queueing: DropTail, RED, RIO, WFQ, SRR, Semantic Packet Queue, REM, Priority, VQ<br>MACs: CSMA, 802.11b, 802.15.4 (WPAN), satellite Aloha | PointToPoint, CSMA, 802.11 MAC low and high and rate control algorithms |
| Physical layer | TwoWay, Shadowing, OmniAntennas, EnergyModel, Satellite Repeater | 802.11a, Friis propagation loss model, log distance propagation loss model, basic wired (loss, delay) |
| Support | Random number generators, tracing, monitors, mathematical support, test suite, animation (nam), error models | Random number generators, tracing, unit tests, logging, callbacks, mobility visualizer, error models |

# A typical NS-3 simulation structure



# Software Integration

NS-3 trace viewed with wireshark:



# NS-3 Simulation Basis

- Simulation time moves in discrete jumps from event to event
- C++ functions schedule events to occur at specific simulation times
- A simulation scheduler orders the event execution
- Simulation::Run() gets it all started
- Simulation stops at a specific time or when all pending events end

# NS-3 Event Scheduling Example

Location: ~/ns-allinone-3.44/ns-3.44/src/core/examples/sample-simulator.cc

http://www.nsnam.org/doxygen/sample-simulator_8cc_source.html

# How to Run Programs

- ./ns3 run scratch/sample-simulator
- ./ns3 run ‘scratch/sample-simulator --RngRun=2’
  - By default, ns-3 simulations use a fixed seed and run number
  - ns3 is the executable

# Network Elements

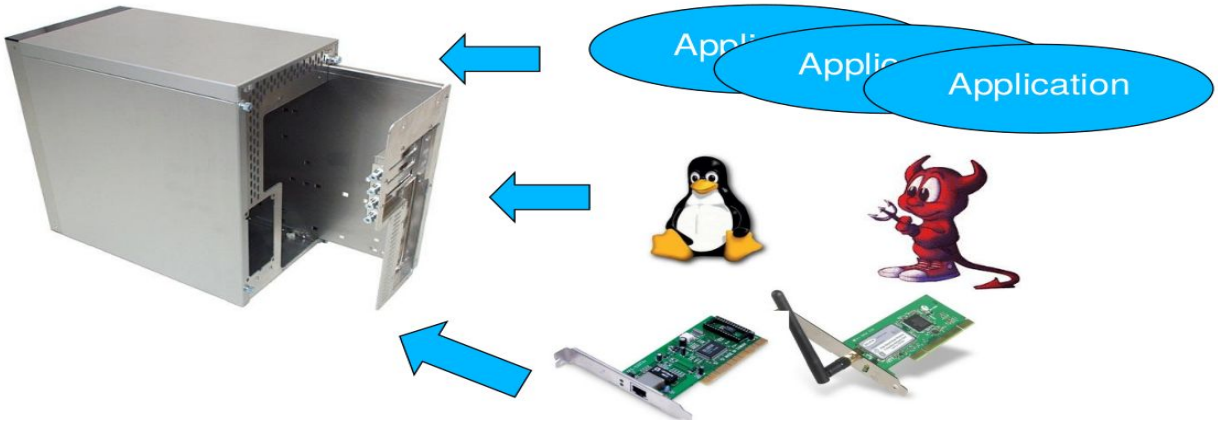
- Basic Components:
- Nodes
- Net Devices (Ethernet NIC)
- Channels
- Application
- Protocol Stack
- Nodes have “Network Devices”
    - Network Devices transfer packets over channels
    - It incorporates L-1 (PHY) & L-2 (MAC)
- The Traffic Control layer in ns-3 is an additional module that intercepts and processes packets between the NetDevices (L2) and any network protocol, such as IP. It's responsible for processing, scheduling, dropping, marking, and policing packets
- Devices interface with L-3 (Network)
- L-3 supports L-4 (Transport)
- L-4 is used by L-5

# Abstractions

- Node
- Application
- Channel
- NetDevice
- Packet
- Topology Helpers – aggregate functionality of modules to make common operations easier than using the low-level API
    - Consists of:
        - container objects
        - helper classes

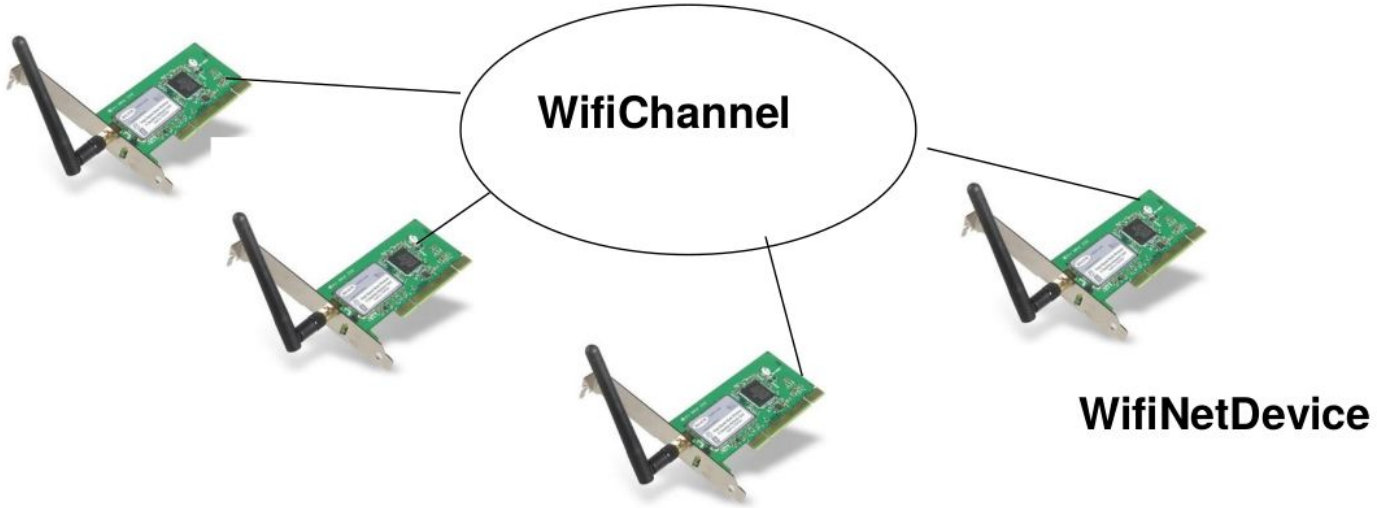
# Fundamentals

- Key objects in the simulator are Nodes, Packets, and Channels
- Node is a husk of a computer to which applications, stacks, and NICs (NetDevs) are added
- Nodes are architected for multiple interfaces

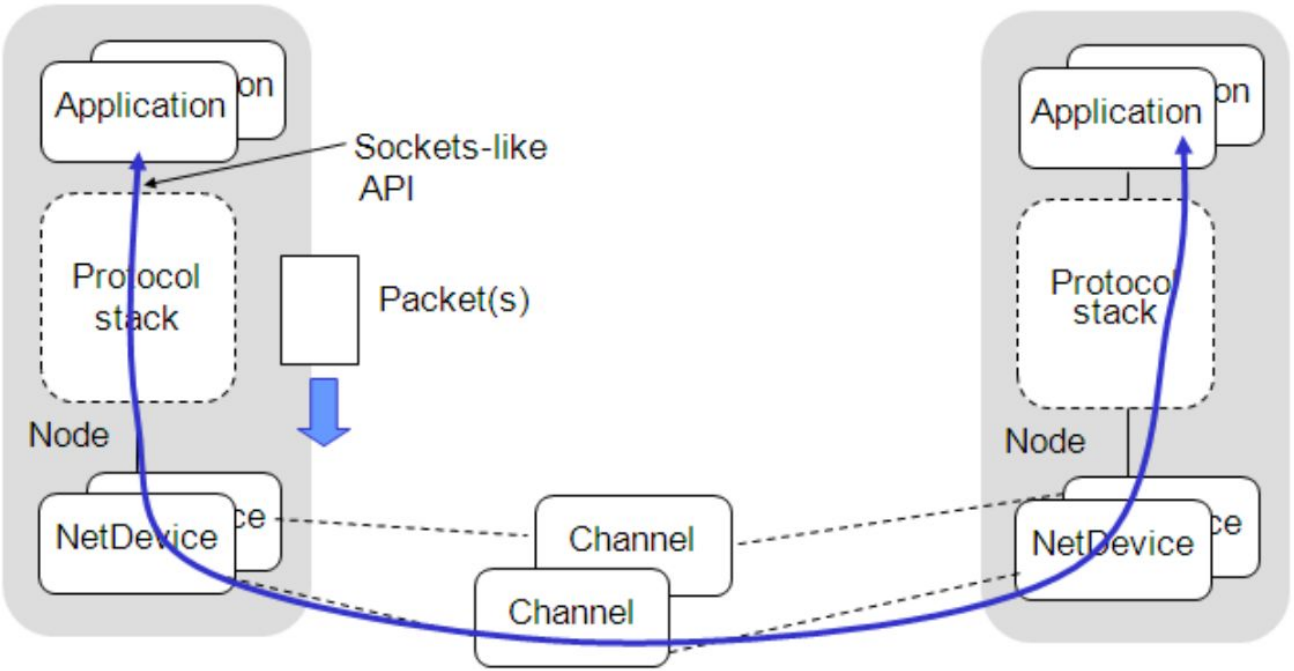


# NetDevices and Channels

NetDevices (NICs) are strongly bound to channels of a matching type



# Basic Model:


Application
Sockets-like
API
Protocol
stack
Packet(s)
Node
NetDevice
Channel
Channel
Application
Protocol
stack
Node
NetDevice

# Structure of NS-3 Program

```
int main (int argc, char *argv[])
{

  // Set default attribute values

  // Parse command-line arguments

  // Configure the topology; nodes, channels, devices, mobility

  // Add (Internet) stack to nodes

  // Configure IP addressing and routing

  // Add and configure applications

  // Configure tracing

  // Run simulation
}
```

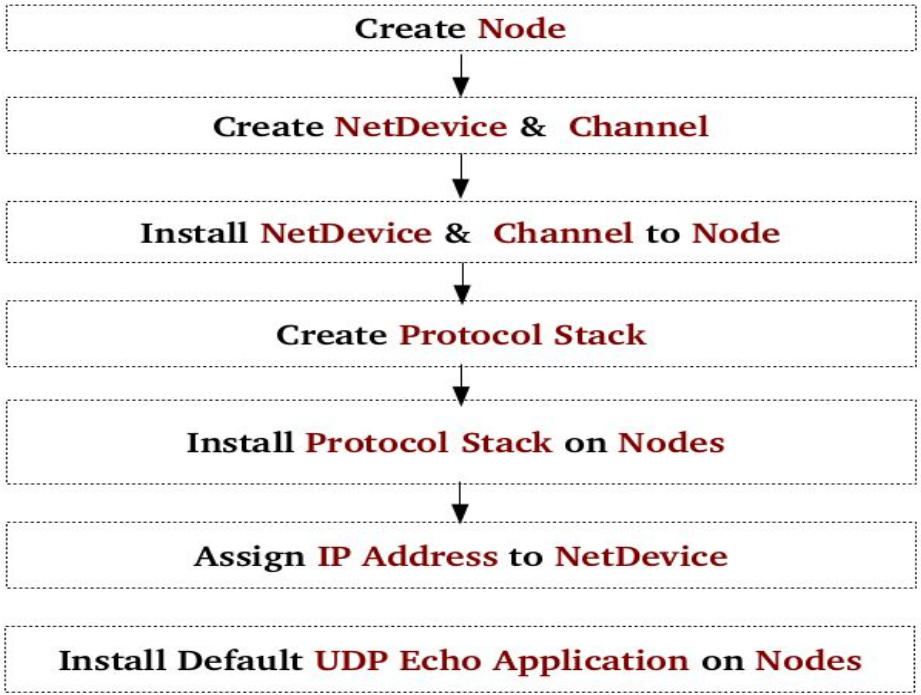
# Network Simulation Example:




- Two nodes, one network interface device per node
- Point-to-point link
  - propagation delay: 2ms, data rate:5 Mbps
- Application

- Udp echo client on node 0, Udp echo server on node 1 on port 9
- Payload size of 1024-byte packet
- time interval between packet is 1 s

# Flow Chart for writing a program in NS-3



# Example Script - 1

```
/* -*- Mode:C++; c-file-style:"gnu"; indent-tabs-mode:nil; -*- */
// GPLv2 Licence …

#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");

int main (int argc, char *argv[])
{
  LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
  LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);

  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices;
  devices = pointToPoint.Install (nodes);
```

include modules that will be used

ns-3 project namespace

enable and disable console message logging by reference to the name

Create Node

Topology Configuration

# Example Script - 2

Install Protocol Stack

Set up Server

Set up Client

```
  InternetStackHelper stack;
  stack.Install (nodes);

  Ipv4AddressHelper address;
  address.SetBase ("10.1.1.0", "255.255.255.0");

  Ipv4InterfaceContainer interfaces = address.Assign (devices);

  UdpEchoServerHelper echoServer (9);

  ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));
  serverApps.Start (Seconds (1.0));
  serverApps.Stop (Seconds (10.0));

  UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);
  echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
  echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
  echoClient.SetAttribute ("PacketSize", UintegerValue (1024));

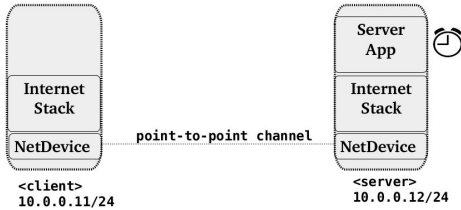
  ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));
  clientApps.Start (Seconds (2.0));
  clientApps.Stop (Seconds (10.0));

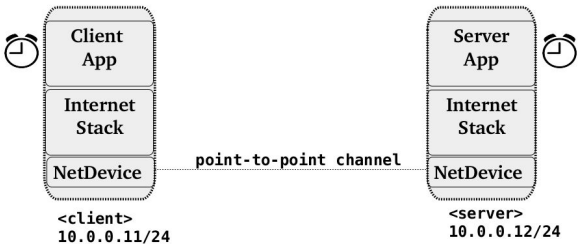
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

Set up internet stack




Set up applications




Run the simulation

# Running Example

Run the commands given below to execute first.cc

- cp examples/tutorial/first.cc scratch/myfirst.cc
- ./ns3 run “scratch/myfirst.cc”
- Output:

```
Consolidate compiler generated dependencies of target scratch_myfirst
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

# Cmd Args

- Add the lines below:
  - int nPackets=0;
  - cmd.AddValue("nPackets", "Number of packets to echo", nPackets);
- ./ns3 run "scratch/myfirst.cc --nPackets=2"
  - This is how one can pass value through CLI to the program.
- ./ns3 run scratch/myfirst.cc --command-template="%s --nPackets=2"
  - Does essentially the same thing in a different way.
- https://www.nsnam.org/docs/release/3.9/tutorial/tutorial_22.html

# Playing with myfirst.cc

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::PointToPointNetDevice"

```
Attributes for TypeId ns3::PointToPointNetDevice
    --ns3::PointToPointNetDevice::Address=[ff:ff:ff:ff:ff:ff]
        The MAC address of this device.
    --ns3::PointToPointNetDevice::DataRate=[32768bps]
        The default data rate for point to point links
    --ns3::PointToPointNetDevice::InterframeGap=[+0ns]
        The time to wait between packet (frame) transmissions
    --ns3::PointToPointNetDevice::Mtu=[1500]
        The MAC-level Maximum Transmission Unit
    --ns3::PointToPointNetDevice::ReceiveErrorModel=[0]
        The receiver error model used to simulate packet loss
    --ns3::PointToPointNetDevice::TxQueue=[0]
        A queue to use as the transmit queue in the device.
```

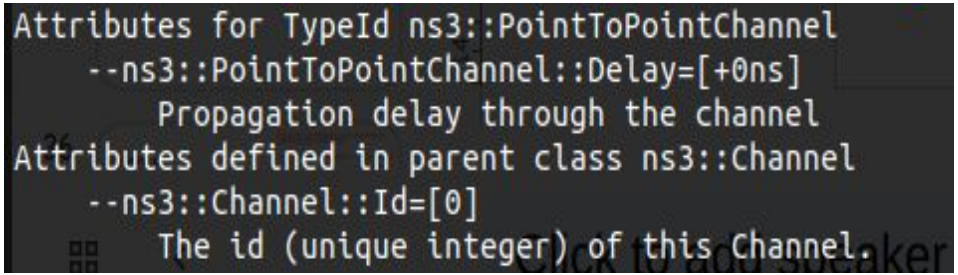
# Playing with myfirst.cc...

Execute: ./ns3 run "scrExecute:

./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::PointToPointChannel"

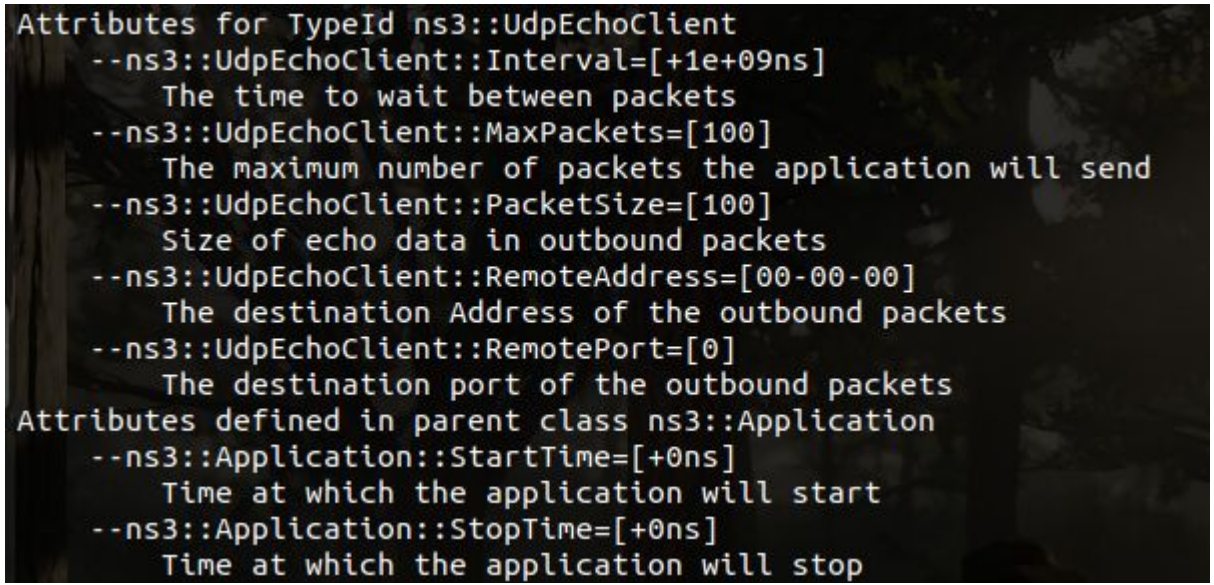
./ns3 run "scratch/myfirst.cc --ns3::PointToPointChannel::Delay=[+0.0ns]"

- Transmission delay through the channel
- Is one of the attributes for PointToPointChannel



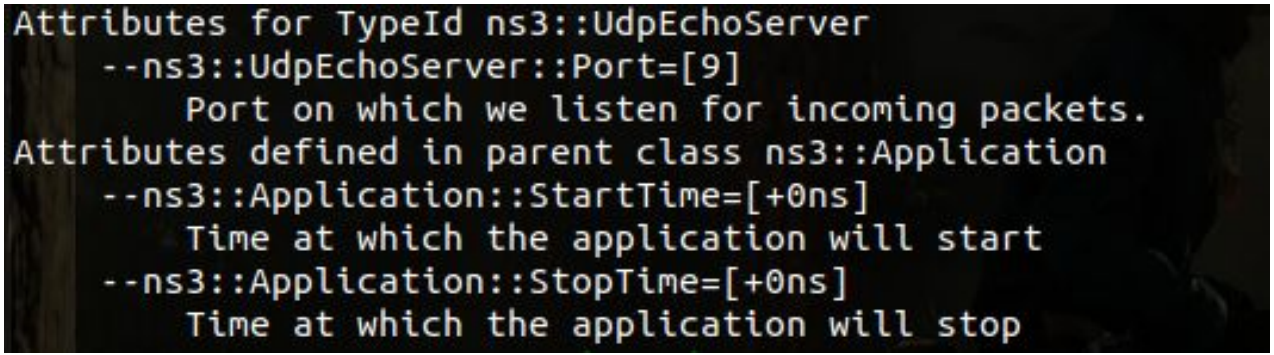
# Playing with myfirst.cc…

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::UdpEchoClient"



# Playing with myfirst.cc…

Execute: ./ns3 run "scratch/myfirst.cc --PrintAttributes=ns3::UdpEchoServer"



# Playing with myfirst.cc…

Execute: ./ns3 run "scratch/myfirst.cc --nPackets=2 --ns3::UdpEchoClient::PacketSize=5000"

Output:

```
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

** Note that if the programmer sets any new value to PacketSize in the program, that overrides the above command line arg value of 5000 Bytes. Same thing even with the nPackets user defined variable.

# Command Line Argument Override

```
CommandLine cmd (__FILE__);
int nPackets=0;
std::cout<<nPackets<<std::endl;
cmd.AddValue("nPackets", "Number of packets to echo", nPackets);
cmd.Parse (argc, argv);
std::cout<<nPackets<<std::endl;
nPackets=7;
std::cout<<nPackets<<std::endl;
```

Output:

```
0
4
7
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
```

# Logs

- Logs are generally used to get useful information.
- NS-3 provides different levels of logs which are as follows:-

– LOG_INFO

– LOG_FUNCTION

– LOG_LOGIC

– LOG_ALL

– etc.

# Enable Logging

- To enable logging :
  - LogComponentEnable(module_name,log_level)
    - Module is any application, UdpEchoClientApplication for instance
  - LogComponentEnableAll(log_level);
    - This enables logging for all the modules
- Example:
  - Module_name
    - ns3::UdpEchoClientApplication
    - ns3::UdpEchoServerApplication
    - And,more
  - Log_level (log level is enabled for the given level and all lower levels)
    - LOG_LEVEL_ERROR
    - LOG_LEVEL_WARN
    - LOG_LEVEL_DEBUG
    - LOG_LEVEL_INFO
    - LOG_LEVEL_FUNCTION
    - LOG_LEVEL_LOGIC
    - LOG_LEVEL_ALL

# Resources

Web site: http://www.nsnam.org

Mailing list: http://mailman.isi.edu/mailman/listinfo/ns-developers

Tutorial: http://www.nsnam.org/docs/tutorial/tutorial.html

Code server: http://code.nsnam.org

Wiki: http://www.nsnam.org/wiki/index.php/Main_Page

Acknowledgements: Mathieu Lacage, Tom Henderson, www.nsnam.org

Training: https://www.nsnam.org/consortium/activities/training/

# Getting Help

./ns3 run "sample-simulator.cc --PrintHelp"

```
sample-simulator [General Arguments]

General Arguments:
    --PrintGlobals:              Print the list of globals.
    --PrintGroups:               Print the list of groups.
    --PrintGroup=[group]:        Print all TypeIds of group.
    --PrintTypeIds:              Print all TypeIds.
    --PrintAttributes=[typeid]:  Print all attributes of typeid.
    --PrintVersion:              Print the ns-3 version.
    --PrintHelp:                 Print this help message.
```

# Our First Code

- **core-module:** Basic ns-3 core classes such as Simulator, Time, Log, etc.
- **network-module:** Provides classes for network components like nodes, devices, and channels.
- **internet-module:** Adds the internet stack to nodes (TCP/IP protocols).
- point-to-point-module: Facilitates the creation of point-to-point links between nodes.
- **applications-module:** Offers application-layer protocols and helper classes (e.g., UDP Echo Client/Server).
- **netanim-module:** Supports animation and visualization of the network simulation using NetAnim.

# Our First Code

- The CommandLine object **cmd** is used to parse command-line arguments.
- The **AddValue** method binds the **tracing** variable to a command-line argument --tracing, allowing users to enable or disable tracing.
- **Parse** processes the actual command-line arguments.
- **NodeContainer nodes:** Creates a container to hold nodes.
- **NodeContainer** is a helper class in ns-3 that manages a collection of Node objects.
- **nodes.Create (2):** Creates two nodes, n0 and n1.

# Our First Code

- **PointToPointHelper pointToPoint:** Helper class to configure and create a point-to-point link.
- **SetDeviceAttribute ("DataRate", StringValue ("5Mbps")):** Sets the data rate of the point-to-point link to 5 Mbps.
- **SetChannelAttribute ("Delay", StringValue ("2ms")):** Sets the propagation delay of the link to 2 ms.
- **NetDeviceContainer devices:** A container to hold the network devices.
- **devices = pointToPoint.Install (nodes):** Installs point-to-point devices and connects them to the nodes, creating a point-to-point link between n0 and n1.

# Our First Code

- **InternetStackHelper stack:** Helper class to install the internet protocol stack (TCP/IP) on nodes.
- **stack.Install (nodes):** Installs the internet stack on both nodes (n0 and n1).
- **Ipv4AddressHelper address:** Helper class to manage IPv4 addresses.
- **address.SetBase ("10.1.1.0", "255.255.255.0"):** Sets the base network address and subnet mask.
- **Ipv4InterfaceContainer interfaces = address.Assign (devices):** Assigns IP addresses to the devices connected by the point-to-point link.

# Our First Code

- **UdpEchoServerHelper echoServer (9):** Creates a UDP echo server application bound to port 9.
- **serverApps = echoServer.Install (nodes.Get (1)):** Installs the server application on node n1.
- **serverApps.Start (Seconds (1.0)):** Starts the server application at 1 second.
- **serverApps.Stop (Seconds (10.0)):** Stops the server application at 10 seconds.
- **UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9):** Creates a UDP echo client application to communicate with the server at IP 10.1.1.2 (address of n1) and port 9.
- **echoClient.SetAttribute ("MaxPackets", UintegerValue (1)):** Configures the client to send only one packet.

# Our First Code

- **echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0))):** Sets the time interval between packets to 1 second.
- **echoClient.SetAttribute ("PacketSize", UintegerValue (1024)):** Sets the packet size to 1024 bytes.
- **clientApps = echoClient.Install (nodes.Get (0)):** Installs the client application on node n0.
- **clientApps.Start (Seconds (2.0)):** Starts the client application at 2 seconds.
- **clientApps.Stop (Seconds (10.0)):** Stops the client application at 10 seconds.

# How Flow Monitor Works

- **Setting Up Flow Monitor:** To use Flow Monitor, you need to include the flow-monitor-module in your simulation script and then use the FlowMonitorHelper class to set it up.
- **Attaching Flow Monitor to Nodes:** The FlowMonitorHelper attaches a Flow Monitor instance to all nodes in the simulation, and it automatically tracks all flows that are created.
- **Flow Classification:** The Flow Monitor uses a FlowClassifier (usually an Ipv4FlowClassifier) to classify packets into flows based on their 5-tuple.
- **Statistics Collection:** The Flow Monitor collects statistics from all flows continuously during the simulation. It gathers metrics such as the number of packets sent/received, the number of bytes sent/received, and computes metrics like throughput, delay, and jitter.
- **End of Simulation:** At the end of the simulation, the statistics are typically printed out or saved to a file for analysis.



# THANK YOU